Distributed Information Systems
Exercise 1

Bilal Kocak / Dominic Malina

March 15, 2026

Abstract

The goal of this exercise is to apply a real-time service with AWS Cloud Infrastructure and learn how Cloud architecture works, how the costs differ between the cloud and on-premise and be familiar with its services.

Question 1: Creating EC2 Instances (4 points)

1. Login to the AWS console

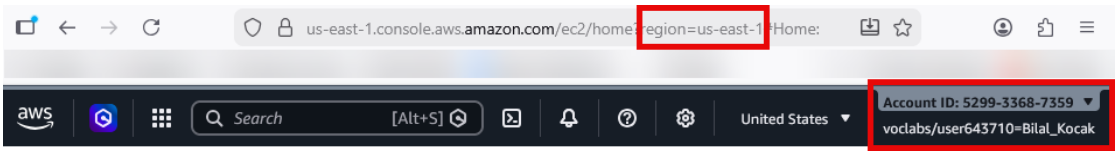


Figure 1: Logged to the AWS console

2. Select the US East (N. Virginia) AWS Region.

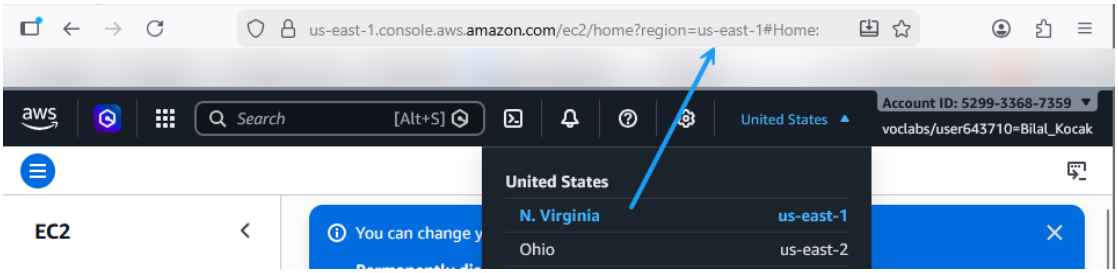


Figure 2: Selected the US East (N. Virginia) AWS Region

3. Select Services → Compute → EC2 on the top menu.

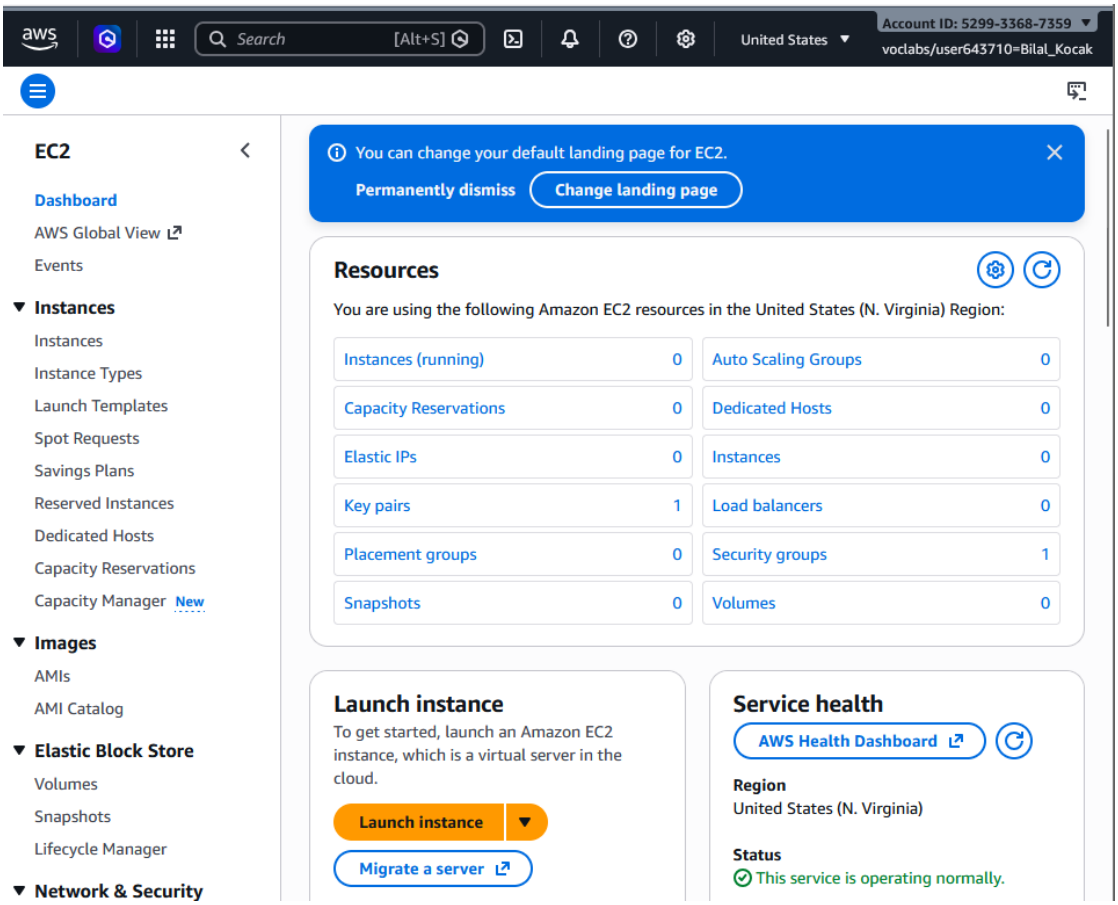


Figure 3: Select Services → Compute → EC2 on the top menu

4. Click on the Launch Instance button.

After *Services* → *Compute* → *EC2* opened, the ***Launch Instance*** button clicked.

5. Select the latest Ubuntu Server AMI.

The latest Ubuntu Server 24.04 LTS AMI was selected. This image uses HVM virtualization and SSD-backed EBS storage. The 64-bit x86 architecture version was chosen because it is compatible with the next selected EC2 t2.micro instance type.

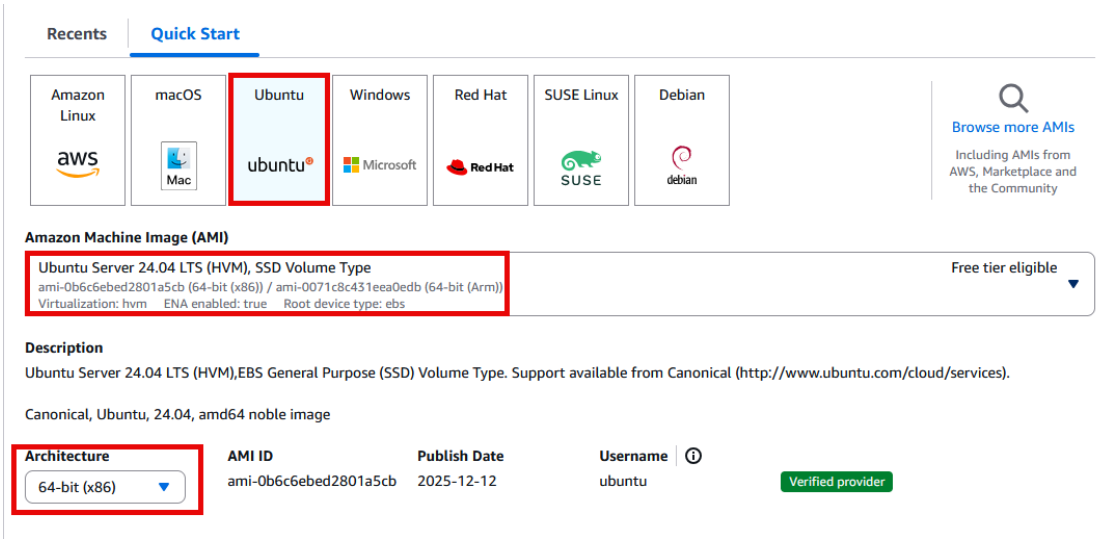


Figure 4: Ubuntu Server Version

6. Select t2.micro instance type.

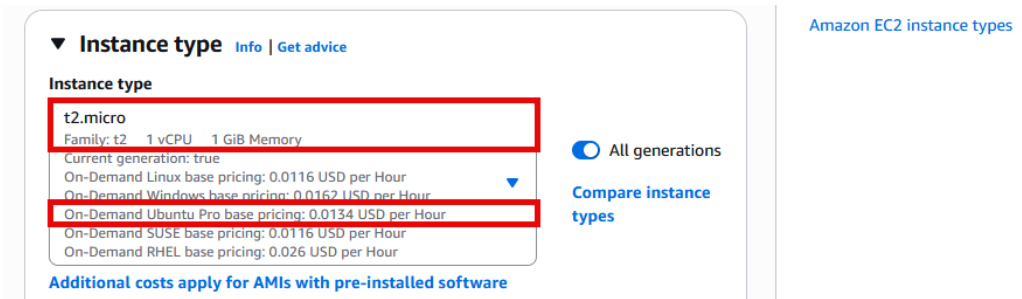


Figure 5: Instance Type

The type of t2.micro instance provides 1 CPU and 1GB RAM. That is enough for our exercise solution. Pricing details are also in the image.

7. Go through the remaining steps of the launch process and try to understand each of the steps and the available options.

In the remaining steps, we have created the KeyPair and Security Group that we need for a secure connection and to define the rules of communication type.

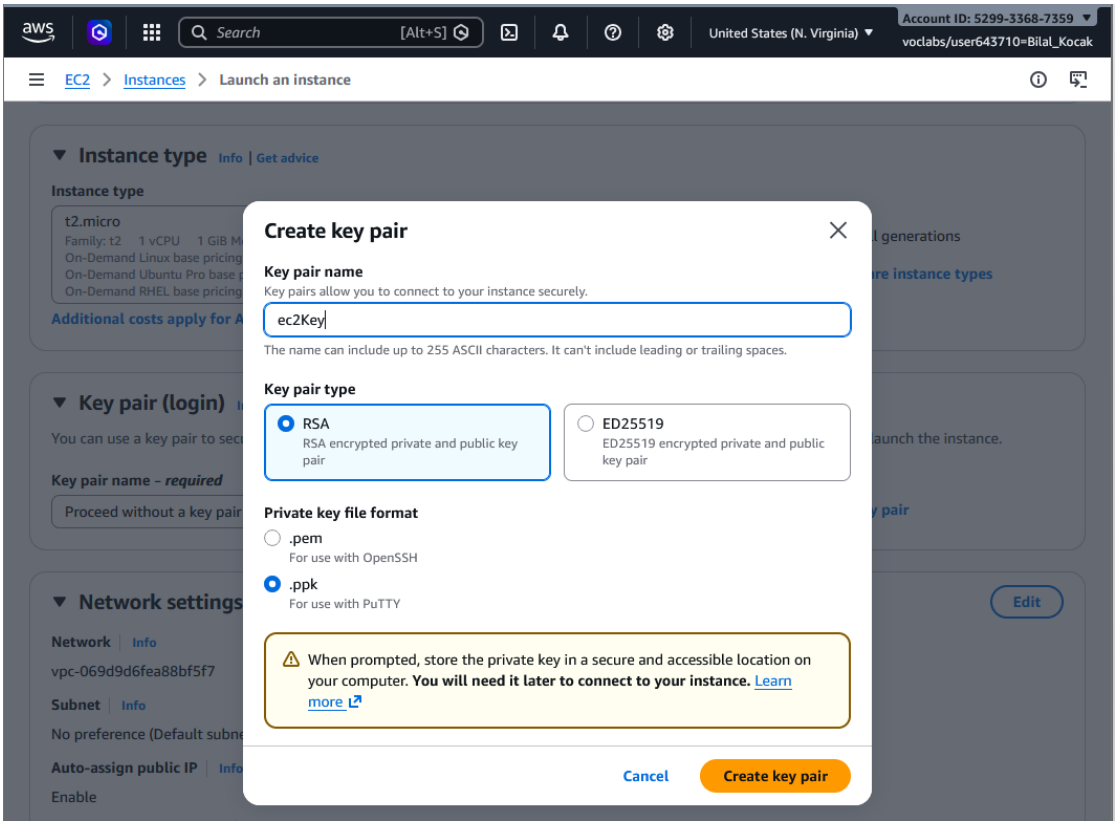


Figure 6: Key-Pair creation

In order to connect the EC2 instance, we need secret keys. We have chosen the RSA type encryption algorithm. We are going to connect using Putty from windows, hence .ppk file format was chosen.

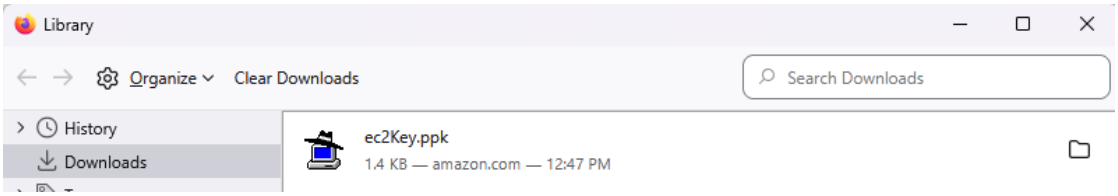


Figure 7: After Key-Pair created, it is automatically downloaded to local PC

This is the file that contains the private key to connect to the EC2 instance with the SSH protocol.

The security group allows us to define which port can have access to the application running on the EC2 instance. As a requirement, we have defined the SSH and HTTP protocols. SSH:22 and HTTP:80 assigned to the defined security group.

After choosing the required properties, we can now create our instance.

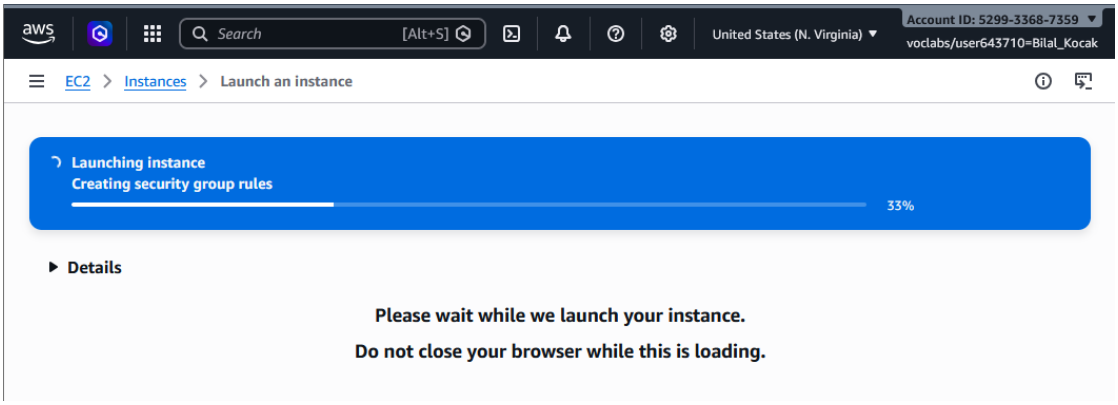


Figure 8: Launching Instance

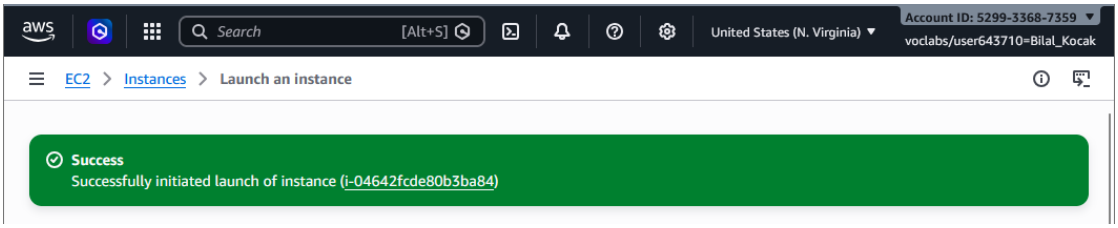


Figure 9: Instance successfully created with unique ID

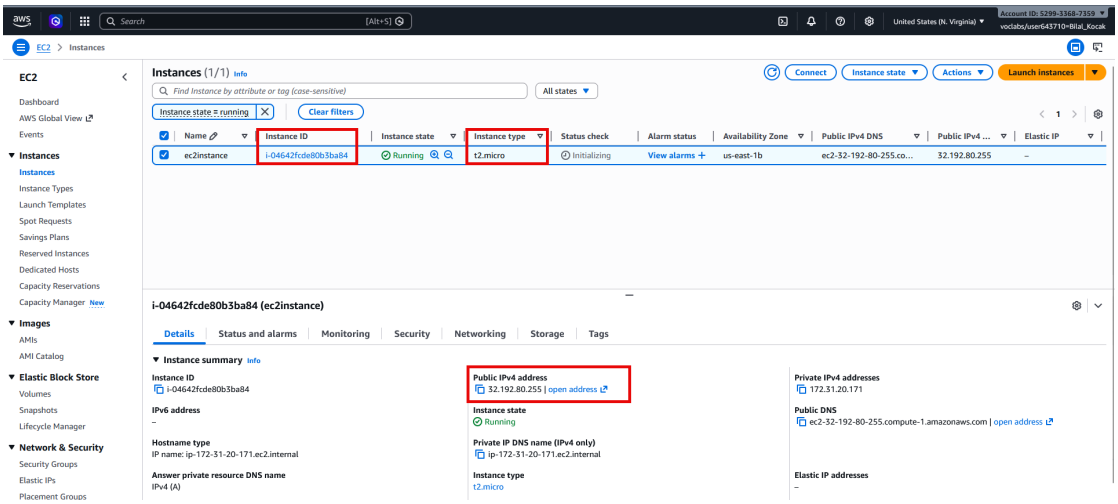


Figure 10: Instance created with a unique ID

Question 2: Managing EC2 Instances (2 points)

1. Connect and login to the previously started EC2 instances using SSH.

Since we have created our instance with .ppk keypair for authentication, we have generated a .pem file from .ppk file that we have, to be able to use ssh protocol for connection.

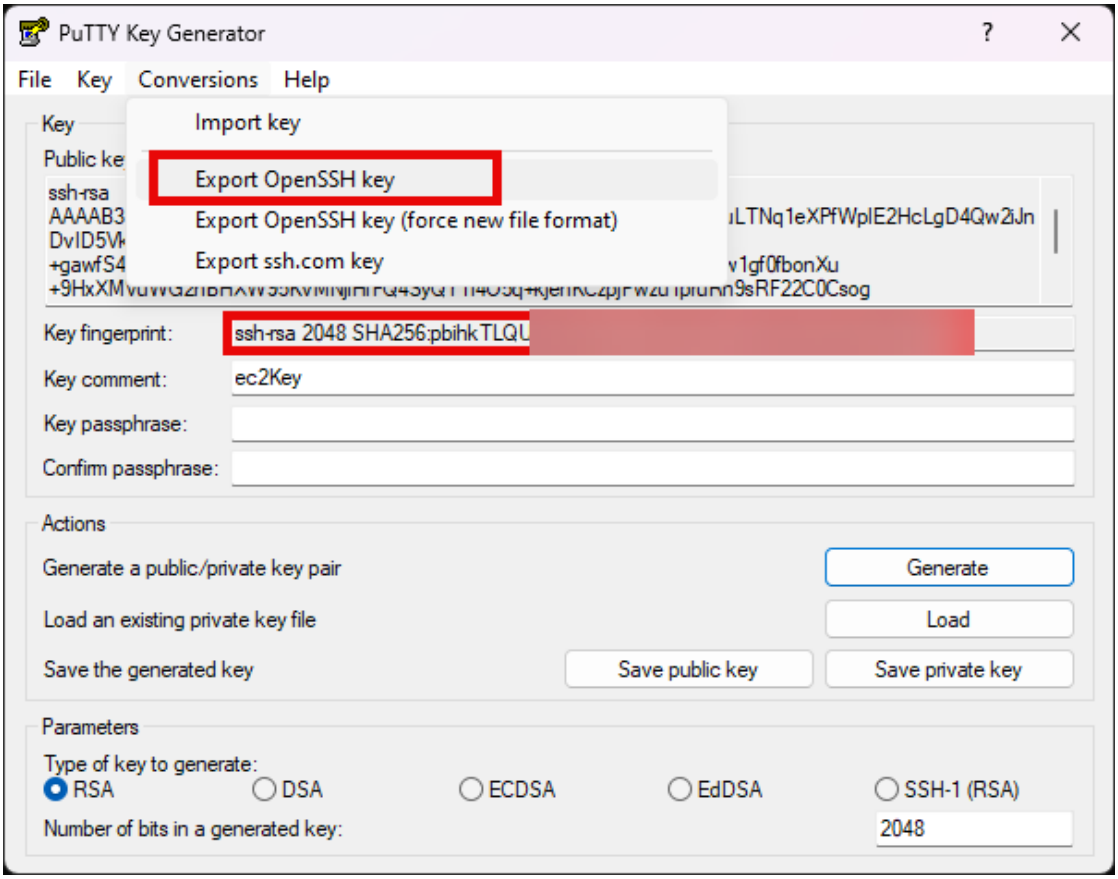


Figure 11: PuttyKey Generator - from .ppk file to .pem file convertion

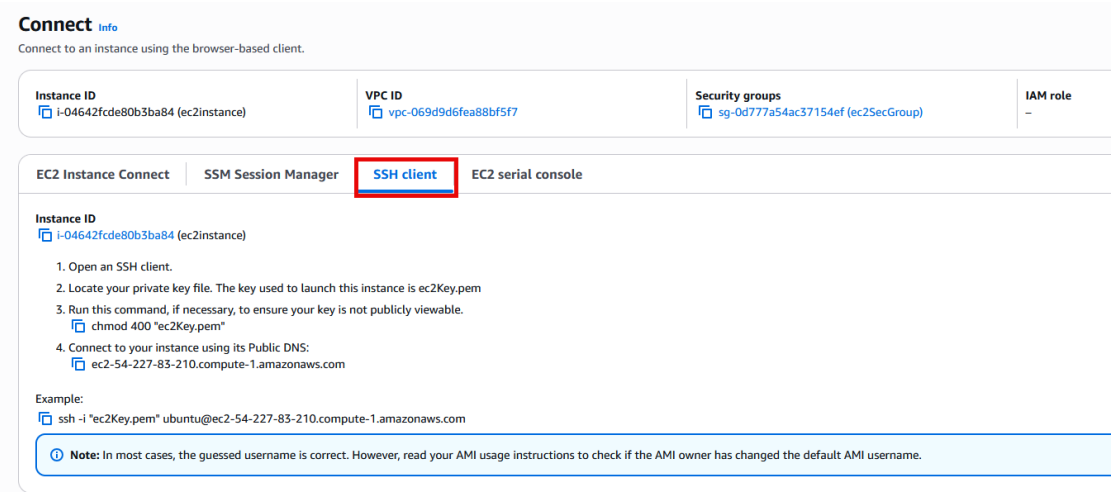


Figure 12: SSH Client

Bash command to connect to an EC2 instance is as follows:

```
ssh -i "C:\path\to\your\my-key.pem" username@your-instance-public-ip
```

- -i stands for "identity file"

- username "ubuntu" for Ubuntu
- IP: Use the Public IPv4 address found in our EC2 dashboard(which we have recognized when we stop and start the Lab, it changes all the time), its a amazon policy to release unused/temporary IP addresses for others' usage.

The name of the instance can also be found on the EC2 connect info page.

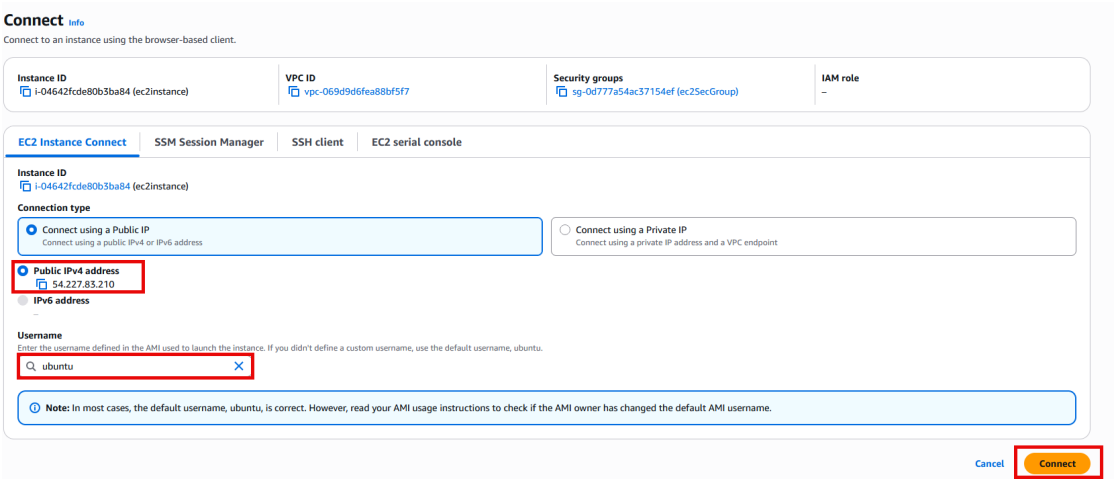
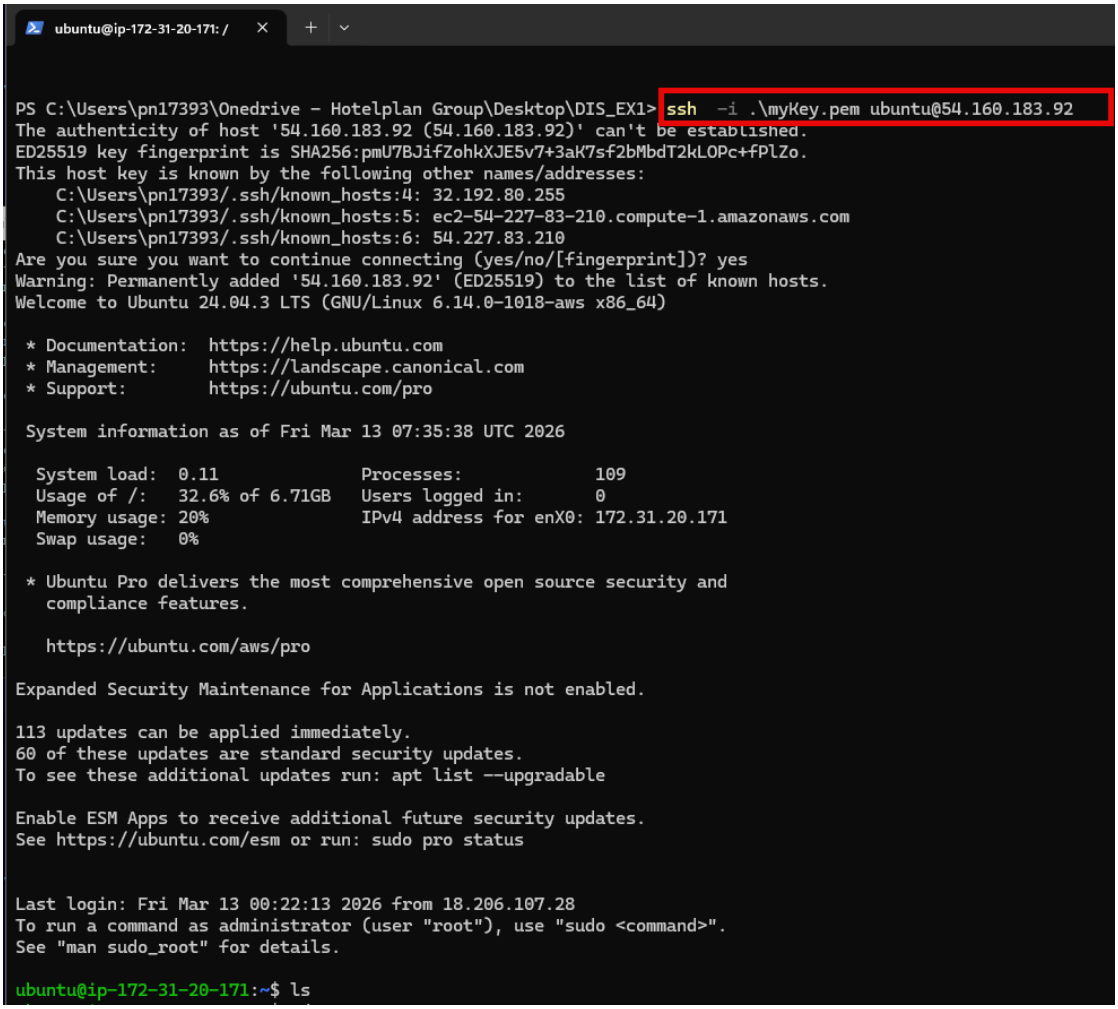


Figure 13: The EC2 Instance Connect



```
ubuntu@ip-172-31-20-171: /

PS C:\Users\pn17393\Onedrive - Hotelplan Group\Desktop\DIS_EX1> ssh  -i .\myKey.pem ubuntu@54.160.183.92
The authenticity of host '54.160.183.92 (54.160.183.92)' can't be established.
ED25519 key fingerprint is SHA256:pmU7BJifZohkXJE5v7+3aK7sf2bMbdT2kLOPc+fPlZo.
This host key is known by the following other names/addresses:
    C:\Users\pn17393/.ssh/known_hosts:4: 32.192.80.255
    C:\Users\pn17393/.ssh/known_hosts:5: ec2-54-227-83-210.compute-1.amazonaws.com
    C:\Users\pn17393/.ssh/known_hosts:6: 54.227.83.210
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '54.160.183.92' (ED25519) to the list of known hosts.
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-1018-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Fri Mar 13 07:35:38 UTC 2026

  System load:  0.11              Processes:             109
  Usage of /:   32.6% of 6.71GB   Users logged in:       0
  Memory usage: 20%               IPv4 address for enX0: 172.31.20.171
  Swap usage:   0%

 * Ubuntu Pro delivers the most comprehensive open source security and
   compliance features.

   https://ubuntu.com/aws/pro

Expanded Security Maintenance for Applications is not enabled.

113 updates can be applied immediately.
60 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


Last login: Fri Mar 13 00:22:13 2026 from 18.206.107.28
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-172-31-20-171:~$ ls
```

Figure 14: SSH connection over the PowerShell

2. Use the AWS console or an IDE extension to explore and document the options for managing your instances.

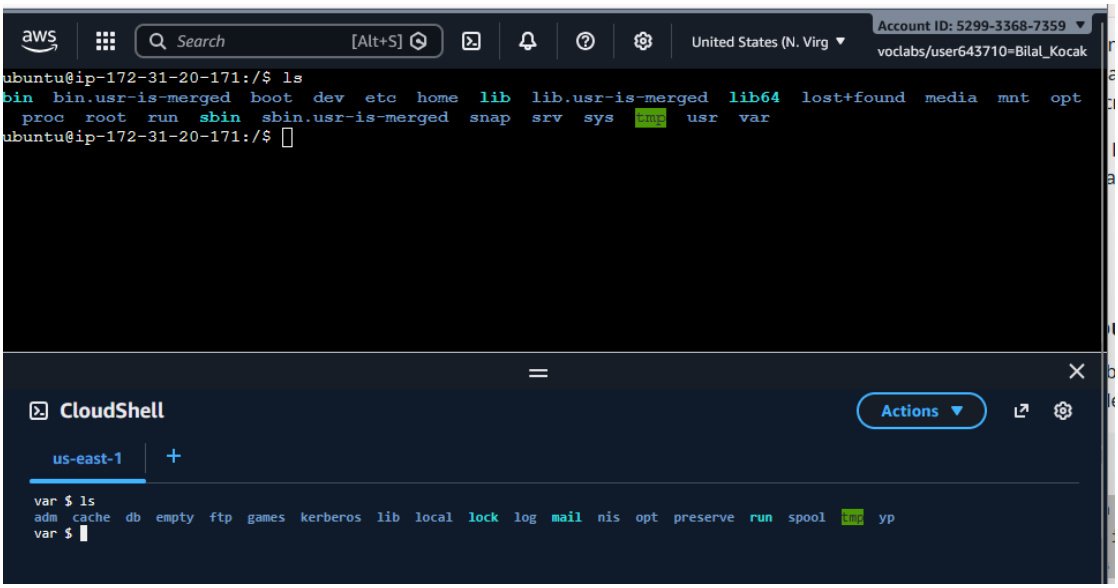


Figure 15: Amazon Terminal and Cloud Shell

Here as well you can connect to the instance over Amazon Terminal and Cloud Shell to manage your applications. The usage of them over SHH, Amazon Terminal, etc., depends on the learner/applicant's preference. All have the same logic that are connect and manages them.

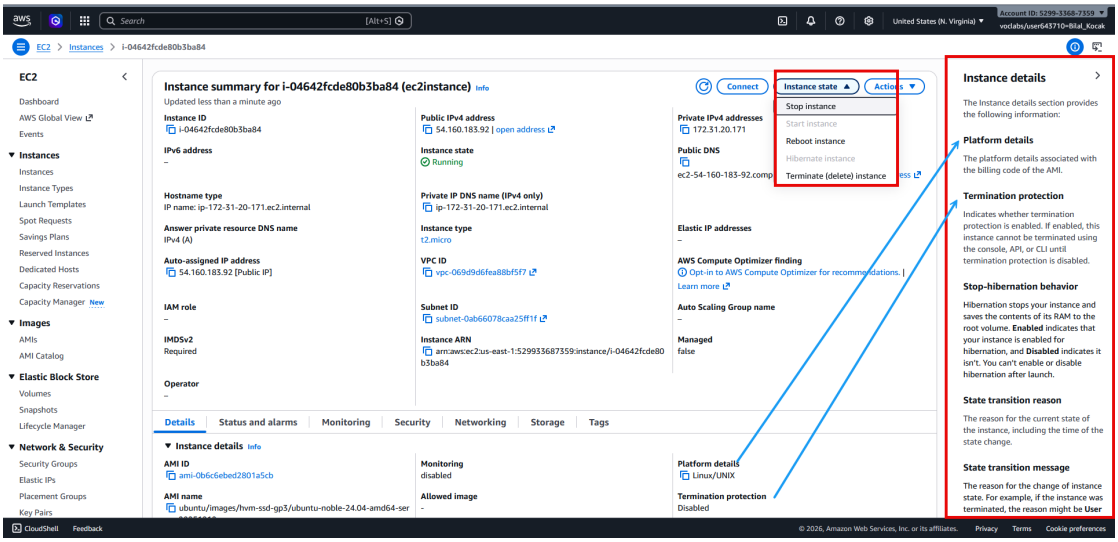


Figure 16: Instance Details

On the Instance summary page, it can be seen that if you want to stop/terminate the instance, options provided how to do so, as shown on the image. Therefore, the instance details of the EC2 can also be seen there with their meaning.

Question 3: Create an AMI (5 points)

1. Create an EC2 Instance as described in Question 1.
   This part is completed.

2. Login to your instance.
This one is also done.

3. Install the Apache Web Server on your EC2 Instance, either via a package management system (apt) or downloading it from the project's website (https://httpd.apache.org).

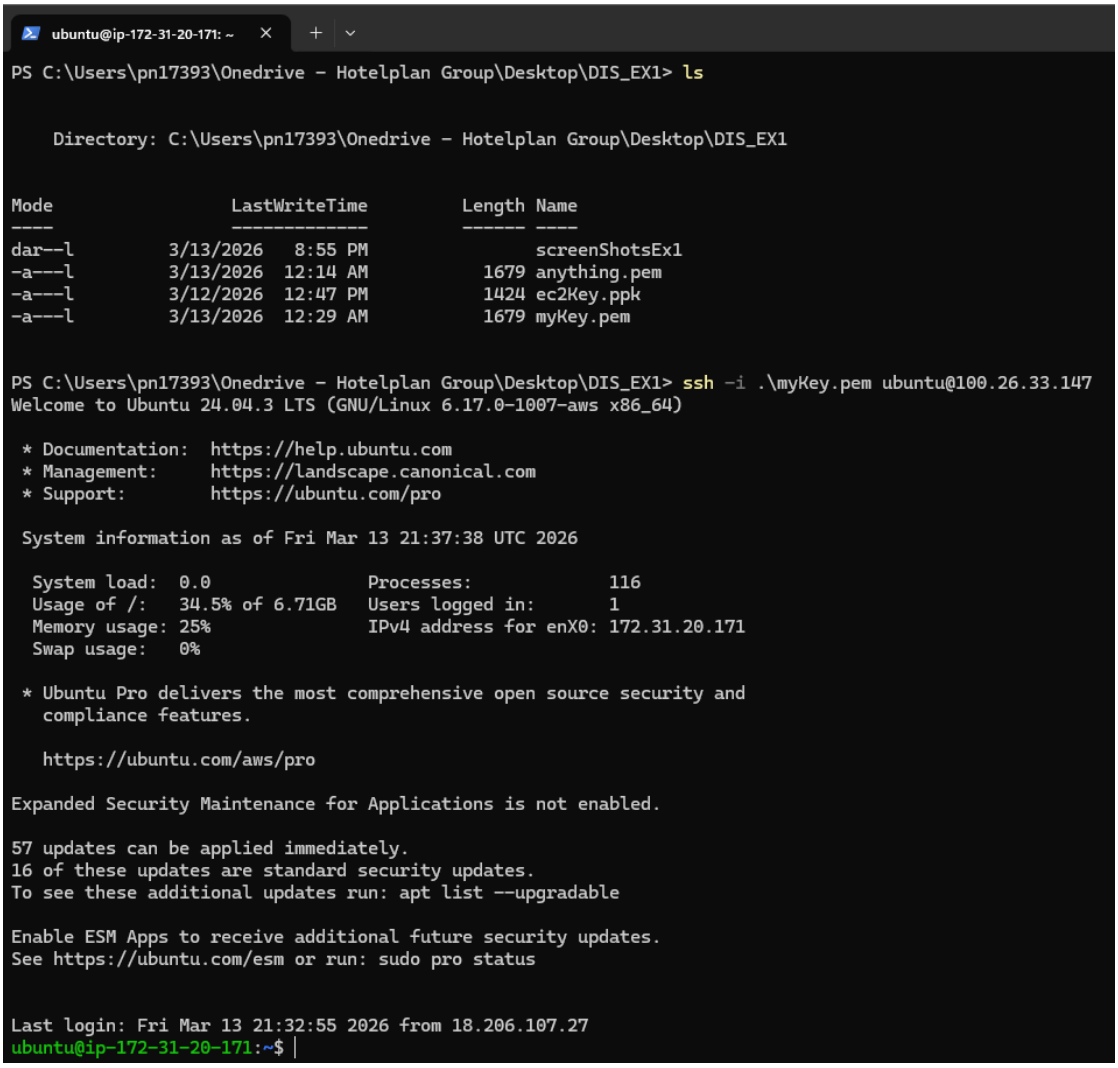


Figure 17: SSH Connect to EC2 Instance

We have successfully connected to our instance via SSH from a local terminal. The next step is to install the Apache HTTP Server which is an open-source web server by the Apache Company. Here we have chosen the apt package management system to install the Apache Web Server.

Updating Package Repository :

```
sudo apt update
```

```
ubuntu@ip-172-31-20-171:~$ sudo apt update
Hit:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Hit:4 http://security.ubuntu.com/ubuntu noble-security InRelease
Get:5 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1815 kB]
Get:6 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main Translation-en [334 kB]
Get:7 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 c-n-f Metadata [16.7 kB]
Get:8 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1566 kB]
Get:9 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe amd64 c-n-f Metadata [33.0 kB]
Get:10 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Packages [2759 kB]
Get:11 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/restricted Translation-en [636 kB]
Get:12 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-backports/main amd64 c-n-f Metadata [368 B]
Get:13 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-backports/universe amd64 c-n-f Metadata [1480 B]
Fetched 7415 kB in 2s (3640 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
58 packages can be upgraded. Run 'apt list --upgradable' to see them.
ubuntu@ip-172-31-20-171:~$
```

Figure 18: Update Package Repository

Install Apache on Ubuntu:

```
sudo apt install apache2
```

```
ubuntu@ip-172-31-20-171: ~
57 packages can be upgraded. Run 'apt list --upgradable' to see them.
ubuntu@ip-172-31-20-171:~$ sudo apt install apache2
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  apache2-bin apache2-data apache2-utils libapr1t64 libaprutil1-dbd-sqlite3 libaprutil1-ldap libaprutil1t64
  liblua5.4-0 ssl-cert
Suggested packages:
  apache2-doc apache2-suexec-pristine | apache2-suexec-custom www-browser
The following NEW packages will be installed:
  apache2 apache2-bin apache2-data apache2-utils libapr1t64 libaprutil1-dbd-sqlite3 libaprutil1-ldap libaprutil1t64
  liblua5.4-0 ssl-cert
0 upgraded, 10 newly installed, 0 to remove and 57 not upgraded.
Need to get 2091 kB of archives.
After this operation, 8113 kB of additional disk space will be used.
Do you want to continue? [Y/n] Y
Get:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 libapr1t64 amd64 1.7.2-3.1ubuntu0.1 [108 k
B]
Get:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/main amd64 libaprutil1t64 amd64 1.6.3-1.1ubuntu7 [91.9 kB]
Get:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/main amd64 libaprutil1-dbd-sqlite3 amd64 1.6.3-1.1ubuntu7 [11
.2 kB]
Get:4 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/main amd64 libaprutil1-ldap amd64 1.6.3-1.1ubuntu7 [9116 B]
Get:5 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/main amd64 liblua5.4-0 amd64 5.4.6-3build2 [166 kB]
Get:6 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 apache2-bin amd64 2.4.58-1ubuntu8.11 [1335
 kB]
Get:7 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 apache2-data all 2.4.58-1ubuntu8.11 [163 k
B]
Get:8 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 apache2-utils amd64 2.4.58-1ubuntu8.11 [98
.2 kB]
```

Figure 19: Install Apache on Ubuntu

Verify that the Apache service:

```
sudo systemctl status apache2
```

```
ubuntu@ip-172-31-20-171:~$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
     Active: active (running) since Fri 2026-03-13 19:57:31 UTC; 1h 58min ago
       Docs: https://httpd.apache.org/docs/2.4/
    Process: 3158 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
    Process: 3230 ExecReload=/usr/sbin/apachectl graceful (code=exited, status=0/SUCCESS)
   Main PID: 3161 (apache2)
      Tasks: 55 (limit: 1117)
     Memory: 6.3M (peak: 6.8M)
        CPU: 465ms
     CGroup: /system.slice/apache2.service
             ├─3161 /usr/sbin/apache2 -k start
             ├─3234 /usr/sbin/apache2 -k start
             └─3235 /usr/sbin/apache2 -k start

Mar 13 19:57:31 ip-172-31-20-171 systemd[1]: Starting apache2.service - The Apache HTTP Server...
Mar 13 19:57:31 ip-172-31-20-171 systemd[1]: Started apache2.service - The Apache HTTP Server.
Mar 13 19:57:59 ip-172-31-20-171 systemd[1]: Reloading apache2.service - The Apache HTTP Server...
Mar 13 19:57:59 ip-172-31-20-171 systemd[1]: Reloaded apache2.service - The Apache HTTP Server.
ubuntu@ip-172-31-20-171:~$
```

Figure 20: Verify that the Apache service

The command output indicates that the service started successfully.

Some more useful commands used:

```
sudo systemctl stop apache2.service
sudo systemctl start apache2.service
sudo systemctl restart apache2.service
sudo systemctl reload apache2.service
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow ssh
sudo ufw allow http
sudo ufw enable (enable firewall)
curl -I localhost
```

Now we need to adjust some configuration of Firewall to allow traffic on port 80.

Available app profiles on UFW (Uncomplicated Firewall):

```
Sudo ufw app list
```

```
ubuntu@ip-172-31-20-171:~$ sudo ufw app list
Available applications:
  Apache
  Apache Full
  Apache Secure
  OpenSSH
ubuntu@ip-172-31-20-171:~$
```

Figure 21: All available app profiles on UFW

We need to allow web traffic on port 80, which is here named Apache.

Allow port 80:

```
sudo ufw allow 'Apache'
```

```
ubuntu@ip-172-31-20-171:~$ sudo ufw allow 'Apache'
Skipping adding existing rule
Skipping adding existing rule (v6)
ubuntu@ip-172-31-20-171:~$
```

Figure 22: Allow web traffic on port 80

Here, it says skipped adding the existing rule, because we have already added it. For the documentation, we run them again to get some screenshots.

Checking UFW status:

```
sudo ufw status
```

```
ubuntu@ip-172-31-20-171:~$ sudo ufw status
Status: active

To                         Action      From
--                         ------      ----
Apache                     ALLOW       Anywhere
22/tcp                     ALLOW       Anywhere
80/tcp                     ALLOW       Anywhere
443                        ALLOW       Anywhere
Apache (v6)                ALLOW       Anywhere (v6)
22/tcp (v6)                ALLOW       Anywhere (v6)
80/tcp (v6)                ALLOW       Anywhere (v6)
443 (v6)                   ALLOW       Anywhere (v6)

ubuntu@ip-172-31-20-171:~$
```

Figure 23: Checking UFW status

Testing locally on the server:

```
curl -I localhost
```

```
ubuntu@ip-172-31-20-171:~$ curl -I localhost
HTTP/1.1 200 OK
Date: Fri, 13 Mar 2026 22:14:41 GMT
Server: Apache/2.4.58 (Ubuntu)
Last-Modified: Fri, 13 Mar 2026 19:52:47 GMT
ETag: "29af-64ced36972aca"
Accept-Ranges: bytes
Content-Length: 10671
Vary: Accept-Encoding
Content-Type: text/html

ubuntu@ip-172-31-20-171:~$
```

Figure 24: Testing Locally on the server

Now we can say the Apache service on Ubuntu is up and running.

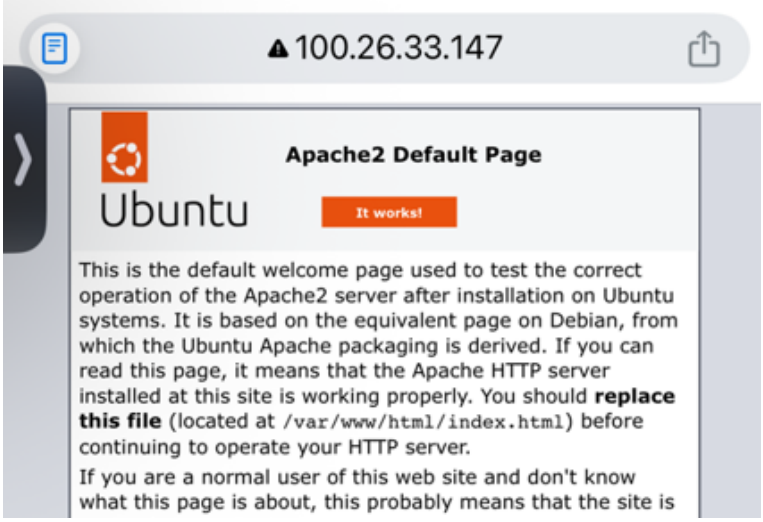


Figure 25: Apache Default Page

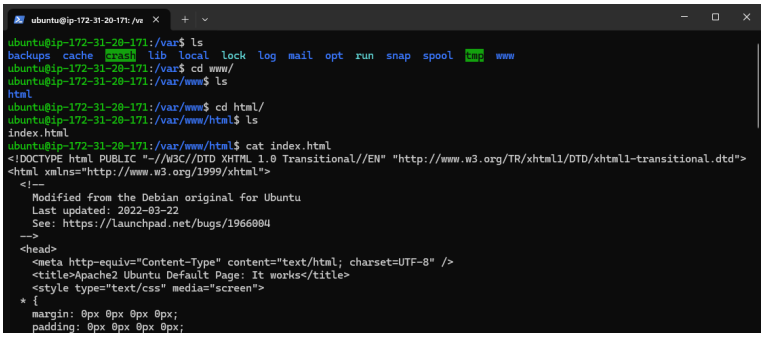


Figure 26: Apache Default Index.html Page during the install of the Apache Server

We need to change the Apache default Index.html with our own.

4. Create a custom index.html file and deploy it on the Web Server in order to check that the installation was successful.

Figure 27: Own index.html file created and deployed to the /var/www/html/

5. Make sure that your website is reachable from the Internet.

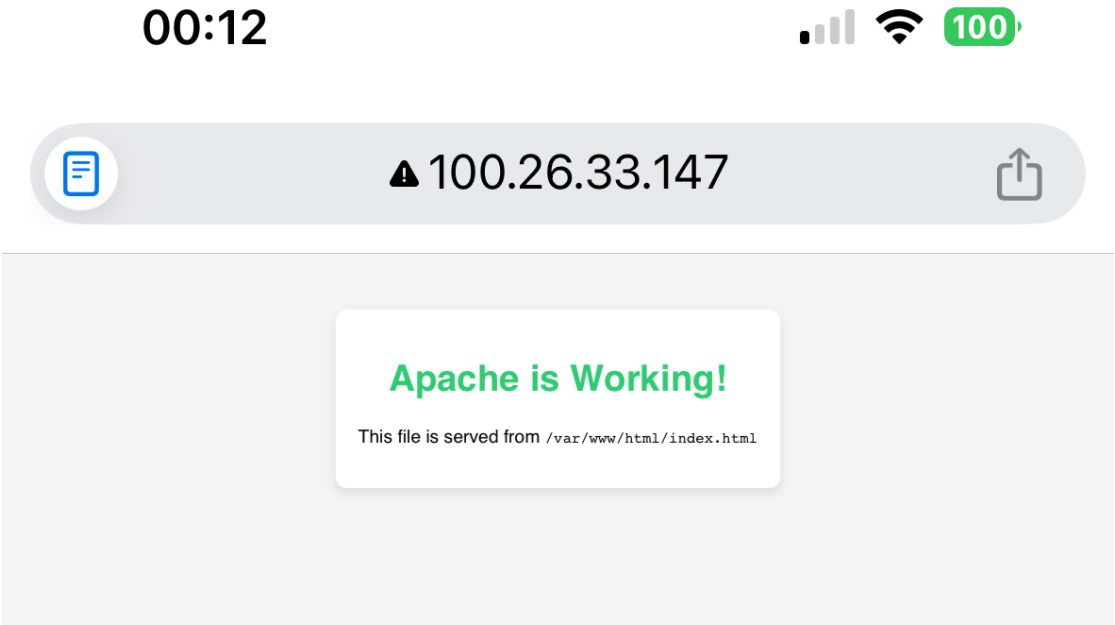


Figure 28: Own index.html is reachable from Internet

6. Create your own AMI.

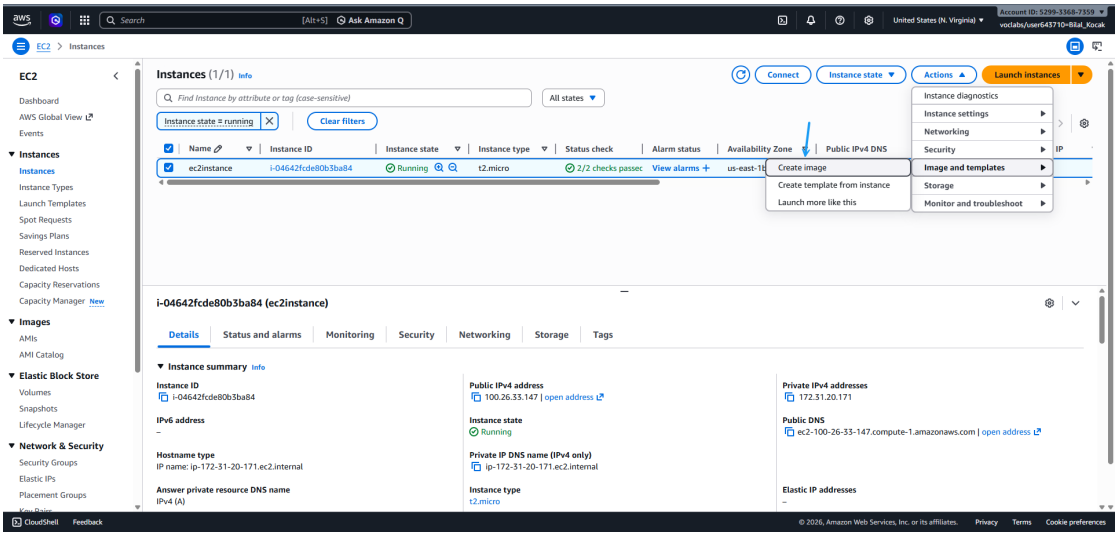


Figure 29: Create Own Image Start

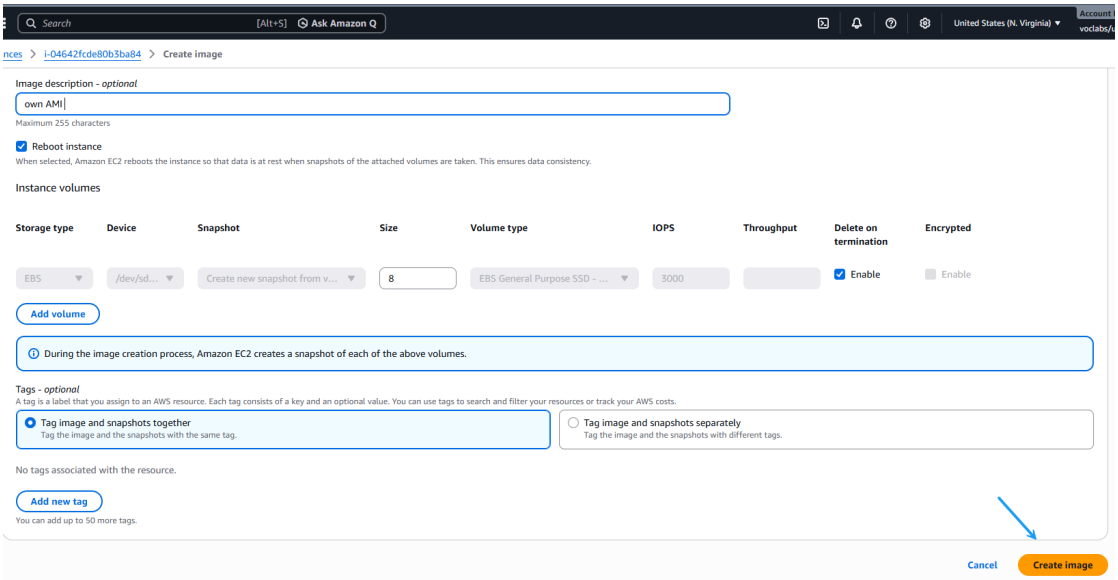


Figure 30: Create Own Image

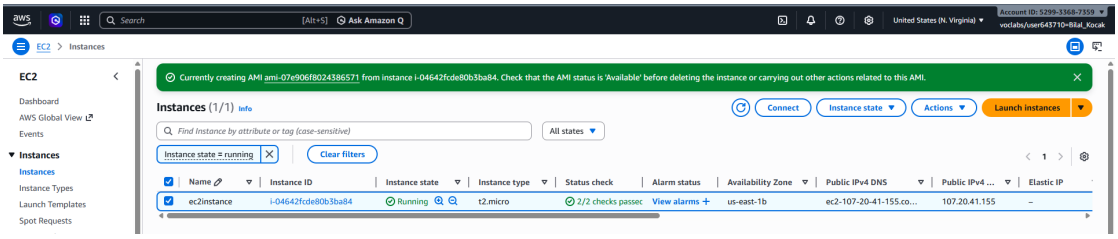


Figure 31: Creating Own Image Progress

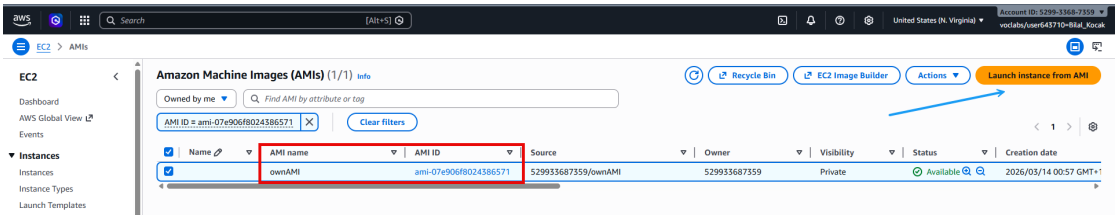


Figure 32: Custom Image Created

7. Start an EC2 instance using your own AMI. The Apache Web Server should be contained in your EC2 instance, as well as in your index.html.

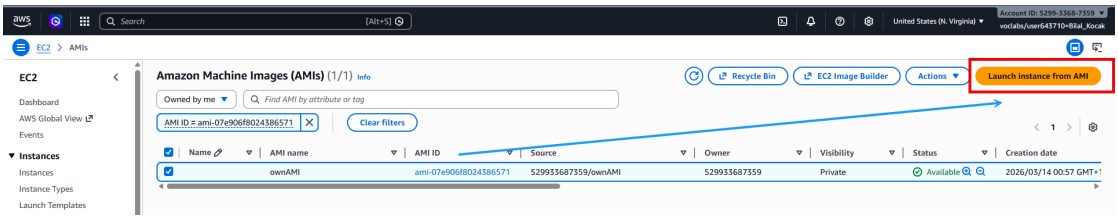


Figure 33: Launching instance from own/custom AMI

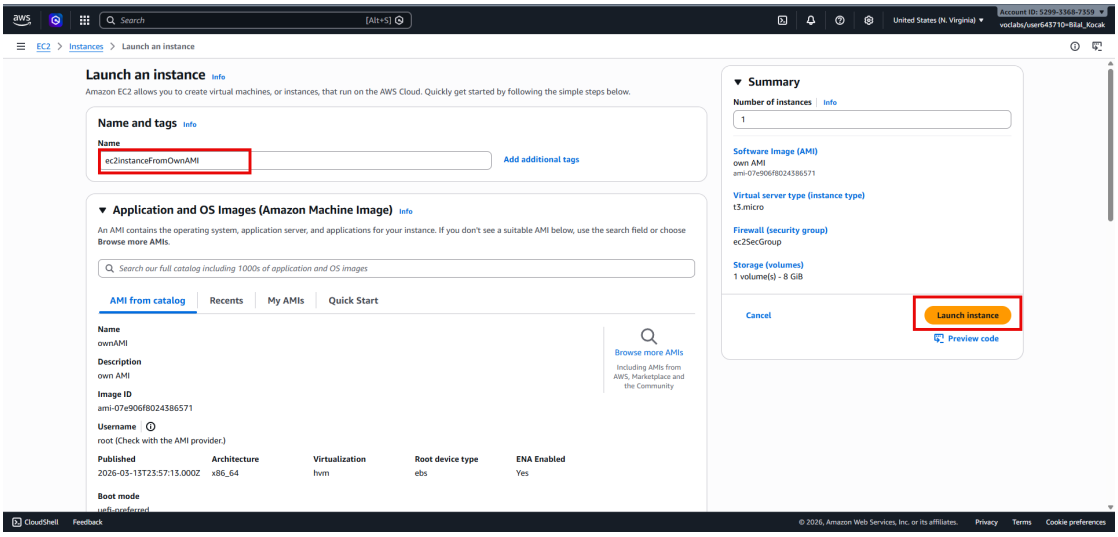


Figure 34: Launching instance from own/custom AMI

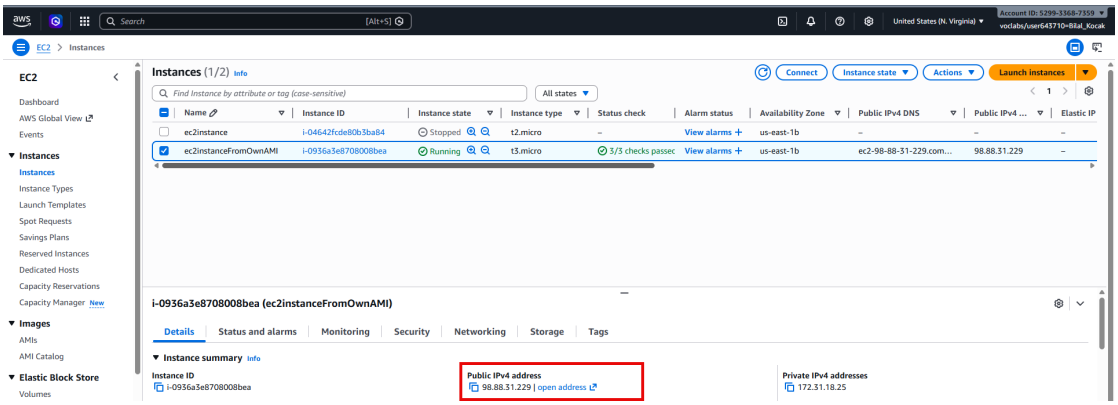


Figure 35: Launching instance from own

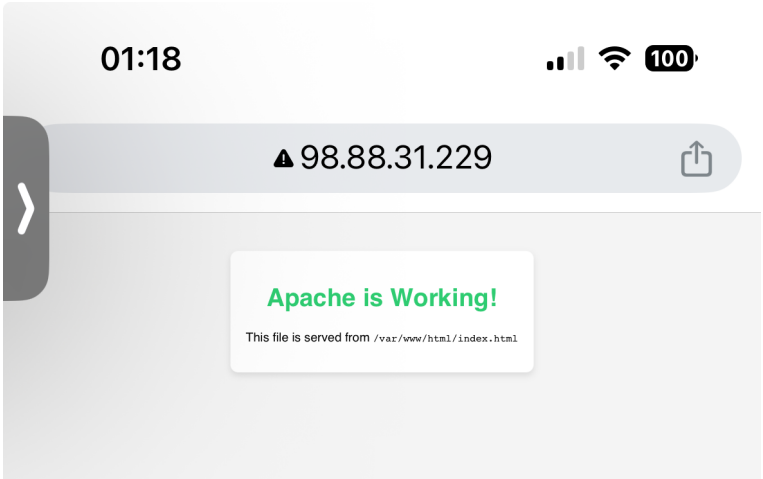


Figure 36: Apache Web Server

It is obviously seen that the Apache Web Server is present in our EC2 Instance as well as our

index.html. That means we can use one single AMI to launch multiple EC2 instances. When a system is configured with all the required software and works very well, you can take a **image** from it as a **snapshot** and then use it in many EC2 instances in case of any failure. It seems like a backup of the system that works fine.

Question 4: Cost Model Comparison (6 points)

Due to the description given to us, the demand/request/traffic for the Online Game Company is mostly stable/smooth. Therefore, for the Travel Agency Company the demand/request/traffic changes suddenly and mostly spiky on December, on other months its stable.

Costs	**Amount**	**Unit**
Hardware		
Server cost (1 server, including 1TB SSD)	4000	$
On-premise network equipment	5000	$
Energy		
Energy consumption for 1 server	750	Watts
Energy cost per kWh	0.40	$
Recurring		
Hardware maintenance cost	15	% of hardware cost per year
1 Gbit/s Internet connection on-premise	100	$/month

Table 1: Upfront and recurring investment costs

Energy consumption for 1 server
700 W / 1000 = 0.75 kW
0.75 kW * 1 hour = 0.75 kWh
0.75 kW * 24 Hours = 18 kWh per day
18 kWh * $0.40 = $7.20 per day
$7.20 * 30 days = $216.00
$216.00 * 12 months = $2,592.00

Energy costs

The Online Game Company always uses **2 servers**. The yearly consumption will be $2,592.00 * 2 = $5,184.00
Total energy cost = $5,184.00

The Travel Agency Company uses **6 servers** in December, but only **1 server** is active throughout all year. So, for all **6 servers** the consumption is $216.00 * 1 month * 6 servers = $1,296.00.
Only **1 server** is active on the rest of the 11 months also, then the price $216.00 * 11 months = $2,376.00. The yearly consumption will be $1,296.00 + $2,376.00 = $3,672.00.
Total energy cost = $3,672.00.

Hardware costs

The Online Game Company always uses **2 servers**.
$4000 * 2 servers = $8000
Network equipment just once $5000 * 1 = $5,000.00
Total hardware cost: $8000 + $5000 = 13,000.00
The Travel Agency Company uses **6 servers**.
$4000 * 6 servers = $24,000.00

Network equipment just once $5000 * 1 = $5,000.00
Total hardware cost: $24000 + $5000 = 29,000.00

Recurring costs

For the Online Game Company Hardware maintenance cost:
$13000 Total hardware cost * 0.15% = $1,950.00
Internet cost: $100 * 12 month = $1,200.00
Total recurring cost: $1,950.00 + $1,200.00 = $3,150.00

For the Travel Agency Company Hardware maintenance cost:
$29000 total hardware cost * 0.15% = $4,350.00
Internet cost: $100 * 12 month = $1,200.00
Total recurring cost: $4,350.00 + $1,200.00 = $5,550.00

Total cost the for Online Game Company: $13,000.00 + $5,184.00 + $3,150.00 = $21,334.00
Total cost for the Travel Agency Company: $29,000.00 + $3,672.00 + $5,550.00 = $38,222.00

Costs	**Amount**	**Unit**
Hardware		
Server cost (1 server, including 1TB SSD)	4000	$
Colocation		
Energy consumption for 1 server	750	Watts
Energy cost per kWh	0.44	$
Service fee	132	$/month
One time setup fee	132	$

Table 2: Costs of using a colocation service for Travel Agency

The colocation cost of 1 year
Hardware cost:$4000 * 6 server = $24,000.00
Energy cost per server: 0.75 kW * 24 Hours * 0.44 * 30 days = $237,6 one month
$237,6 * 1 month * 6 servers = $1,425.6
$237,6 * 11 months * 1 servers = $2,613.6
Total = $1,425.6 + $2,613.6 = $4,039.2 Service fee: 132 * 12 = $1,584.00
One time setup: $132
Total cost of colocation = $24,000.00 + $4,039.2 + $1,584.00 + $132 = 29,754.8

Question a: Financial Analysis and Recommendations

- **Private Gaming Company (Fixed Workload)** Based on the calculations, the Private Gaming Company has a fixed workload of 2 servers.
  - Break-even Point: There is a clear break-even point at approximately Year 3. Before this point, AWS is cheaper due to zero upfront costs. After Year 3, the cumulative cost of AWS surpasses the On-Premise cost.
  - Recommendation: I recommend the On-Premise solution. Since the workload is constant and predictable, the company benefits from lower long-term operating costs once the initial hardware investment is amortized.
  - On-Premise Advantages:
    * Full Control: Complete ownership of the physical hardware and data security protocols.
    * Long-term Cost Efficiency: Lower total cost of ownership.

    * Predictability: No fluctuating monthly bills based on data transfer or minor usage changes.

- **Travel Agency (Variable Workload)**
  - Break-even Point: There is no break-even point within the 10-year period. The AWS solution remains significantly cheaper from Year 1 through Year 10.
  - Recommendation: We strongly recommend the AWS Cloud solution. The agency benefits from Elasticity, paying for 6 servers only when needed (December). Buying 6 physical servers for a 1-month peak is a waste of capital.

Question b: Colocation and Unpredictability//

- **Colocation vs. AWS for Travel Agency**
  - Analysis: Based on the 10-year chart, there is a break-even point at Year 6. Previously, AWS was significantly cheaper. After Year 6, Colocation becomes cheaper than AWS because the initial hardware investment is fully amortized.
  - Recommendation: Despite the long-term crossover, AWS is still the better recommendation. This choice is dependent on the project's lifecycle and risk tolerance. For a 10-year period, AWS is superior because it avoids a massive upfront CAPEX and the risk of maintaining 6 servers that sit idle for 92% of the year. Furthermore, physical hardware typically becomes obsolete by Year 5 or 6, meaning the "savings" in Colocation would likely be erased by the need to purchase new servers.
- **What if the workload is highly unpredictable?** If the workload becomes highly unpredictable, I would strongly recommend the Cloud (AWS) solution over On-Premise or Colocation for the following reasons:
  - Scalability: In a Cloud environment, you can spin up dozens of servers in minutes to meet a sudden surge in traffic and shut them down just as quickly.
  - Risk Mitigation: In On-Premise or Colocation, if traffic suddenly triples, your website crashes because you don't have enough physical servers. If traffic drops to zero, you are still stuck paying for the hardware and space.
  - Pay-as-you-go: Cloud allows the company to turn "uncertainty" into a manageable operational expense that scales exactly with demand

Question 5: Services (3 points)

EC2 (Elastic Compute Cloud) - IaaS EC2 provides raw virtualized infrastructure. So, the user has full control over the operating system, middleware, and software stack, it represents a classic Infrastructure as a Service model

S3 (Simple Storage Service) – IaaS It acts as foundational storage infrastructure that allows users to store, analyze and manage data without managing the underlying physical hardware.

Elastic Beanstalk - PaaS It enables customers to deploy applications by simply uploading their code, while the service automatically handles infrastructure provisioning, capacity scaling, load balancing, and health monitoring.

SES (Simple Email Service) — SaaS/PaaS SES is primarily classified as SaaS because it provides a fully functional software application for emails. There is no need e.g. to manage SMPT servers. SES also exhibits strong PaaS characteristics, it is designed for developers. It provides APIs and SDKs that serve as a "platform"

RDS (Relational Database Service) - PaaS RDS automates undifferentiated database management tasks, such as provisioning, configuring, backing up, and patching. User manages the database schema and data so it's a ready to use platform.

CloudWatch – SaaS CloudWatch is a Software as a Service tool used for monitoring. It is a ready-to-use monitoring solution that provides dashboards and alarms. User only use the software to gain insights into their resources.